

ui.badge 全面详解（基于 NiceGUI 文档）

ui.badge 是 NiceGUI 中用于展示标签、计数或状态标识的轻量级组件，基于 Quasar 的 QBadge 组件实现。其核心价值在于通过简洁的视觉样式突出关键信息（如数字计数、状态标签、分类标识），支持灵活的颜色定制、图标搭配和交互扩展，适用于各类界面的信息标注场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：**基于 Quasar 的 QBadge 组件，继承其紧凑布局、圆角样式和颜色体系。
- 核心功能：**支持文本 + 图标组合展示、自定义背景色 / 文本色、尺寸调整，可作为独立组件或嵌套在其他组件（如按钮、卡片）中使用。
- 交互支持：**默认无交互逻辑，可通过绑定事件实现点击、悬停等交互效果（2.22.0+ 版本支持 `on_click`）。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
text	str	徽章的文本内容（支持纯文本、数字或简短标识）	-
color	str / None	徽章背景色（支持 Quasar 颜色、Tailwind 颜色、CSS 颜色）	'primary'
text_color	str / None	徽章文本颜色（支持 Quasar/Tailwind/CSS 颜色，优先级高于背景色，默认文本色）	None
icon	str / None	徽章左侧显示的图标名称（符合 Quasar 图标规范）	None
outline	bool	是否使用轮廓样式（仅显示边框，无填充背景）	False
rounded	bool	是否使用圆形样式（仅适用于单个字符 / 数字的徽章）	False
size	str	徽章尺寸（可选值：'xs'、'sm'、'md'、'lg'、'xl'，对应 Quasar 尺寸体系）	'md'

二、基础使用示例

1. 最简徽章（纯文本 / 数字）

适用于计数展示（如未读消息数）、状态标签（如“已完成”）等场景：

```
from nicegui import ui

with ui.row().classes('gap-4'):
    # 数字计数徽章（默认尺寸和颜色）
    ui.badge('5', color='red')
    # 文本状态徽章（自定义蓝色背景）
    ui.badge('Active', color='blue')
    # 轮廓样式徽章（无填充，仅边框）
    ui.badge('Pending', outline=True, color='orange')
    # 圆形徽章（仅适用于单个字符）
    ui.badge('!', rounded=True, color='purple')

ui.run()
```

效果：四个徽章横向排列，分别展示数字、文本、轮廓和圆形样式，颜色各不相同。

2. 带图标的徽章

通过 `icon` 参数添加图标，增强视觉识别性，适用于分类标签、功能标识等场景：

```
from nicegui import ui

with ui.row().classes('gap-4'):
    # 图标+文本组合（通知图标+未读数量）
    ui.badge('12', icon='notifications', color='red')
    # 仅图标徽章（无文本）
    ui.badge(icon='warning', color='yellow', text_color='black')
    # 轮廓+图标徽章
    ui.badge('New', icon='star', outline=True, color='green')

ui.run()
```

效果：徽章左侧显示图标，右侧（或无）显示文本，图标与文本颜色自动适配背景色（可通过 `text_color` 手动调整）。

3. 嵌套在其他组件中

徽章可嵌套在按钮、卡片、列表项等组件中，实现关联信息标注（如按钮操作的未处理数量）：

```

from nicegui import ui

with ui.column().classes('gap-4'):
    # 嵌套在按钮中（右上角徽章）
    with ui.button('Messages'):
        ui.badge('3', color='red').classes('absolute top-1 right-1')

    # 嵌套在卡片中（顶部标签）
    with ui.card().classes('relative w-48 h-32'):
        ui.badge('Hot', icon='fire', color='red').classes('absolute top-2 left-2')
        ui.label('Popular Item').classes('mt-8 text-center')

ui.run()

```

效果：按钮右上角显示未读消息数徽章，卡片左上角显示“Hot”标签徽章，通过 `absolute` 定位实现悬浮效果。

三、核心功能与进阶用法

1. 动态修改属性（文本、颜色、图标）

通过组件实例的方法动态更新徽章的文本、颜色、图标等属性，适用于数据实时变化的场景（如实时计数更新）：

```

from nicegui import ui

# 创建徽章并赋值给变量
badge = ui.badge('0', icon='like', color='blue')

# 动态增加计数
def increment():
    current = int(badge.text)
    badge.set_text(str(current + 1))
    # 计数超过 10 时改变颜色
    if current + 1 > 10:
        badge.set_color('red')

# 动态修改图标
def change_icon():
    badge.set_icon('favorite' if badge.icon == 'like' else 'like')

# 控制按钮
with ui.row().classes('mt-4 gap-2'):
    ui.button('Increment', on_click=increment)
    ui.button('Change Icon', on_click=change_icon)

ui.run()

```

效果：点击“Increment”按钮，徽章计数 + 1，超过 10 后变为红色；点击“Change Icon”按钮，切换徽章图标。

2. 交互功能（点击、悬停）

2.22.0+ 版本支持 `on_click` 事件，结合 `tooltip` 可实现带交互的徽章（如点击徽章查看详情）：

```
from nicegui import ui

# 带点击事件和悬停提示的徽章
badge = ui.badge('5 Unread', icon='mail', color='indigo')
badge.tooltip('Click to view unread messages') # 悬停提示
badge.on_click(lambda: ui.notify('Viewing 5 unread messages')) # 点击事件

ui.run()
```

效果：鼠标悬停徽章显示提示文本，点击徽章触发通知。

3. 数据绑定（动态同步属性）

通过 `bind_*` 系列方法，将徽章的文本、颜色等属性与数据对象绑定，实现数据变化时自动更新界面：

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.unread_count = 3
        self.is_important = False

state = AppState()

# 绑定文本（未读计数）和颜色（是否重要）
badge = ui.badge(
    text=str(state.unread_count),
    icon='alert',
    color='red' if state.is_important else 'blue'
).bind_text(state, 'unread_count') \
  .bind_color(state, 'is_important', converter=lambda x: 'red' if x else 'blue')

# 控制绑定属性的按钮
with ui.row().classes('mt-4 gap-2'):
    ui.button('+1', on_click=lambda: setattr(state, 'unread_count', state.unread_count + 1))
    ui.switch('Important', value=state.is_important).bind_value(state, 'is_important')

ui.run()
```

效果：点击“+1”按钮，徽章文本自动同步未读计数；切换“Important”开关，徽章颜色自动切换为红色 / 蓝色。

4. 样式定制（尺寸、间距、自定义 CSS）

通过 `size` 参数、`classes` 类或 `style` 直接设置 CSS，实现精细化样式调整：

```
from nicegui import ui
```

```
with ui.row().classes('gap-4'):
    # 自定义尺寸（超小、超大）
    ui.badge('XS', size='xs', color='gray')
    ui.badge('XL', size='xl', color='black')

    # 自定义内边距和圆角（Tailwind 类）
    ui.badge('Custom Padding', classes='px-4 py-2 rounded-full', color='teal')

    # 自定义文本颜色和字体大小（内联 CSS）
    ui.badge('Big Text', style='font-size: 18px; color: white;', color='orange')

ui.run()
```

效果：徽章尺寸、内边距、字体大小和颜色均自定义，满足不同界面设计需求。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
enabled	BindableProperty	是否启用（禁用后无法触发事件）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
icon	BindableProperty	图标名称（可绑定数据动态修改）
text	BindableProperty	文本内容（可绑定数据动态修改）
color	BindableProperty	背景色（可绑定数据动态修改）
text_color	BindableProperty	文本颜色（可绑定数据动态修改）
outline	bool	是否为轮廓样式（只读，初始化时设置）
rounded	bool	是否为圆形样式（只读，初始化时设置）
size	str	尺寸（可动态修改，支持 'xs'/'sm'/'md'/'lg'/'xl'）
visible	BindableProperty	是否可见（可绑定数据）

2. 关键方法

(1) 状态控制

- `disable()`：禁用徽章（无法触发点击事件）
- `enable()`：启用徽章
- `set_enabled(value: bool)`：设置启用状态（True/False）
- `set_visibility(visible: bool)`：设置徽章可见性

(2) 属性修改

- `set_text(text: str)`: 动态修改文本内容
- `set_icon(icon: str | None)`: 动态修改图标
- `set_color(color: str)`: 动态修改背景色
- `set_text_color(text_color: str)`: 动态修改文本颜色
- `set_size(size: str)`: 动态修改尺寸 (需为支持的尺寸值)
- `update()`: 触发客户端界面更新 (修改属性后需手动调用, 绑定数据时无需)

(3) 事件与绑定

- `on_click(callback)`: 绑定点击事件 (2.22.0+ 版本支持)
- `on(type: str, handler)`: 订阅任意 DOM 事件 (如 `mouseover`、`mousedown`)
- `bind_text(target_object, target_name)`: 双向绑定文本到目标对象属性
- `bind_color_from(target_object, target_name, converter=None)`: 单向绑定背景色 (支持转换器)
- `bind_visibility(target_object, target_name)`: 双向绑定可见性

(4) 其他实用方法

- `tooltip(text: str)`: 为徽章添加悬停提示
- `delete()`: 删除徽章组件
- `mark(*markers)`: 添加标记 (用于测试或元素查询)
- `style(css: str)`: 直接设置内联 CSS 样式

五、使用场景与注意事项

1. 适用场景

- 计数展示: 未读消息数、通知数、购物车商品数等数字类信息。
- 状态标签: 任务状态 (已完成 / 待处理)、数据类型 (热门 / 推荐)、权限标识 (管理员 / 普通用户) 等。
- 分类标注: 列表项、卡片、按钮等组件的分类或属性标注 (如 “新品”“优惠” 标签)。
- 交互入口: 带点击事件的徽章 (如点击查看详情、清除通知)。

2. 关键注意事项

- 文本长度限制: 徽章设计为轻量级组件, 建议文本长度控制在 1-5 个字符 (数字 / 简短文本), 过长会导致样式变形。
- 圆形样式限制: `rounded=True` 仅适用于单个字符 / 数字, 多字符文本使用圆形样式会导致显示异常。
- 颜色优先级: `text_color` 参数优先级高于背景色默认文本色 (如深色背景默认白色文本, 设置 `text_color='black'` 后强制显示黑色)。
- 版本兼容性: `on_click` 方法仅在 2.22.0+ 版本支持, `html_id` 仅在 2.16.0+ 版本支持, 使用时需确认版本。
- 嵌套定位: 嵌套在其他组件中时, 需通过 `absolute` 定位 (结合父组件 `relative`) 实现悬浮效果, 避免影响父组件布局。

六、进阶示例：实时通知徽章（结合定时器）

结合 `ui.timer` 实现实时更新的通知徽章，模拟动态计数场景（如实时消息推送）：

```
from nicegui import ui
import random

class NotificationState:
    def __init__(self):
        self.count = 0

state = NotificationState()

# 绑定状态的徽章
badge = ui.badge(
    text=str(state.count),
    icon='notifications',
    color='red',
    rounded=True
).bind_text(state, 'count')

# 定时器：每 3-5 秒随机增加 1-3 条通知
def simulate_notifications():
    if state.count < 99: # 限制最大计数为 99
        state.count += random.randint(1, 3)
    else:
        state.count = 99 # 超过 99 显示为 99
        ui.notify('Too many notifications!')

# 启动定时器（随机间隔 3-5 秒）
ui.timer(random.uniform(3, 5), simulate_notifications, repeat=True)

# 清除通知按钮
ui.button('Clear', on_click=lambda: setattr(state, 'count', 0)).classes('mt-4')

ui.run()
```

效果：徽章计数每 3-5 秒随机增加，超过 99 时触发提示，点击“Clear”按钮可重置计数为 0，全程自动同步界面。